

Adentro de Widevine L1 en Windows: El despacho de tres capas que nadie mapeó, y las ranuras por-cuadro que nunca disparan

Jeremy Erazo (trexnegr0)
Investigador independiente de seguridad
mendozayt13@gmail.com

Junio de 2026

Resumen ejecutivo. El CDM Widevine L1 que viene en todo navegador de la familia Chromium en Windows ha sido, hasta ahora, una caja negra para cualquiera fuera de Google. Este paper la abre. Tres vtables concéntricas, diecinueve ranuras públicas de CDM_11, verificadas byte-a-byte en dos builds de navegadores distintos. Un mapa empírico runtime-confirmado de qué ranuras se ejercen realmente durante streaming hardware-decoded real. Una distribución de ofuscación estilo MBA a lo largo del espacio de ranuras que refleja la profundidad del parsing de datos atacante-controlables. Tablas de alcanzabilidad tanto para Netflix como para una fuente pública Widevine no-Netflix. Y, de fondo, una metodología que captura todo esto sin jamás leer un byte de contenido, una clave de contenido, ni un IV desde la memoria del CDM. Los defensores se llevan una tabla de superficie de ataque por ranura más firmas de detección; los atacantes se llevan la desambiguación de cuáles ranuras tienen que vivirles. La metodología completa se libera con fuente junto al paper; no se libera artefacto de extracción ni exploit weaponizado.

Resumen

El Módulo de Descifrado de Contenido (CDM) Widevine viene prácticamente en todo navegador de escritorio mayor y soporta el camino de reproducción de Encrypted Media Extensions (EME) para todo servicio comercial de streaming. Los internals del build L1 de Windows están documentados sólo fragmentariamente: los archivos de cabecera describen el ABI de C++ que expone al navegador anfitrión, pero el binario en sí es código cerrado, parcialmente ofuscado, y se comporta diferente dependiendo de si el anfitrión delega la decodificación a pipelines protegidos por hardware. Este paper presenta una ingeniería inversa arquitectural estilo Buchanan de la implementación CDM_11 tal como viene en Chrome 149 y Edge 150 sobre Windows 11. Identificamos un *modelo de despacho de tres capas* (*thunks* de la vtable externa, una vtable intermedia `m_impl` de *wrappers* limpios y *double-trampolines*, y cuerpos de *handlers* por ranura), localizamos cada capa byte-a-byte en ambos binarios, y caracterizamos empíricamente cuáles ranuras se invocan durante reproducción en steady-state vs. cuáles permanecen dormidas. Confirmamos en runtime que durante reproducción hardware-decoded de Netflix las ranuras por-cuadro del CDM_11 (`Decrypt`, `DecryptAndDecodeFrame`, `DecryptAndDecodeSamples`) no disparan mientras una tabla de despacho singleton interna genera ~2,300 llamadas por segundo de trabajo cripto/scheduler invisible desde el límite EME. Documentamos la distribución de ofuscación estilo mixed-boolean-arithmetic en el espacio de ranuras y mostramos que se concentra donde el CDM empieza a parsear datos estructurados atacante-controlables, mientras está ausente en ranuras que pasan *blobs* opacos. Después verificamos la alcanzabilidad de las ranuras de gestión-de-sesión desde dos aplicaciones

EME distintas (Netflix y una fuente pública de prueba Widevine) y mostramos comportamiento de despacho idéntico. El paper sitúa este trabajo en dos décadas de investigación previa de DRM, recorre los precedentes legales y metodológicos, y documenta tanto los caminos exitosos como los falsos arranques que caracterizaron el proceso de ingeniería inversa. Ningún clave de contenido, buffer de cuadro, ni byte de ciphertext se captura en ningún momento del estudio.

1. Introducción

Modern desktop video DRM is the bridge between commercial streaming services and the browser's media pipeline. When a user plays a video in Netflix, Amazon Prime, Disney+ or any other Widevine-protected source, the browser invokes Encrypted Media Extensions (EME) to delegate the cryptographic operations to a Content Decryption Module (CDM). In Chromium-based browsers on Windows, that CDM is the proprietary `widevinecdm.dll` library shipped by Google [1]. The browser loads the library inside a Mojo utility process (`-utility-sub-type=media.mojom.`

`CdmServiceBroker`), connects to it over IPC, and asks the library to construct a CDM instance implementing a stable C++ interface defined in `content_decryption_module.h` [2]. Desde el punto de vista del anfitrión, la interfaz es chica (diecinueve métodos virtuales) y bien documentada: `Initialize`, `GetStatusForPolicy`, `CreateSessionAndGenerateRequest`, `UpdateSession`, y otros doce métodos que cubren el intercambio de licencias, el descifrado, la decodificación y el lifecycle.

Desde la perspectiva de la *biblioteca*, esta interfaz limpia de diecinueve métodos es la punta visible de un iceberg mucho mayor. El build L1 de Windows de Widevine implementa un runtime ofuscado de varios millones de instrucciones bajo decenas de megabytes de código cerrado, se integra con Windows Protected Media Path (PMP) cuando hay decodificación por hardware disponible, y está sujeto a un régimen de licencia estricto que prohíbe el estudio de sus internals de descifrado. El conocimiento público sobre la implementación es fragmentario: el árbol de Chromium contiene el *lado anfitrión* del cableado EME, varios proyectos académicos y de aficionados han estudiado la variante *Android L3* [19, 21], pero la variante L1 de escritorio en Windows ha recibido comparativamente poca atención arquitectural en la literatura abierta.

La motivación de este estudio es tanto metodológica como técnica. Varios episodios legales recientes han mostrado que *publicar lo que uno aprende* sobre internals de DRM está más expuesto legalmente que *aprenderlo*, y que la reproducibilidad de un resultado pesa tanto como el resultado en sí [15, 16, 18]. Siguiendo el modelo Buchanan de ingeniería inversa arquitectural [10, 11] — explorar todo, documentar la técnica, no publicar artefacto de extracción — nos propusimos construir una metodología completamente reproducible y completamente abierta para estudiar el CDM Widevine y aplicarla a las preguntas que son publicables sin cruzar la línea de la circumvención de gestión de derechos digitales.

Lo nuevo de este trabajo

1. **El primer mapa arquitectural público de L1 `widevinecdm.dll` en Windows.** Tres vtables concéntricas, cada entrada clasificada, cada offset de despacho verificado byte-a-byte. Validación cruzada en Chrome 149 y Edge 150. Reproducible drop-in sobre cualquier build futuro vía el discoverer libre de parámetros que liberamos.
2. **Inactividad bajo decode por hardware, runtime-confirmada.** Cuatro ranuras de la outer-vtable patcheadas en un solo experimento. A lo largo de más de 90 segundos de reproducción 4K HDR continua de Netflix, `g_calls = 0` en la ranura 14 (`DecryptAndDecodeFrame`), la 15 (`DecryptAndDecodeSamples`) y la 9 (`Decrypt`) simultáneamente. El trabajo por-cuadro en L1 no atraviesa la interfaz pública del CDM; la superficie de ataque vista desde EME para intercepción por-cuadro es, en términos prácticos, vacía.
3. **Una tabla de superficie de ataque por ranura.** Qué ranuras ingieren input estructurado atacante-controlable (PSSH, respuesta de licencia, política), qué ranuras pasan blobs opacos, qué ranuras están

dormidas. Consistente entre fuentes: Netflix y el demo Widevine de Bitmovin ejercen las mismas ranuras en el mismo orden. Los defensores usan la tabla directamente para alcance de detección; los atacantes la usan directamente para priorizar los handlers de capa-3 que de verdad importan.

4. **Ofuscación estilo MBA, mapeada relativa a la profundidad.** Cada una de las diecinueve ranuras CDM_11 caracterizada en cada capa del despacho. La ofuscación vive a profundidades variables, apareciendo consistentemente donde los datos estructurados del atacante empiezan a ser parseados y ausente consistentemente en ranuras de passthrough. Ejemplos concretos con firmas byte-level para cada una.
5. **Una metodología que viaja.** El patrón de *observación shape-only* es generalizable a cualquier binario closed-source con una interfaz estable. El mismo diseño de hook aplica, sin cambiar de forma, al `PlayReadyHost` de PlayReady, al `AppleSpringboard` de FairPlay, o a cualquier otro contrato host-CDM donde la pregunta es *qué entradas disparan cuándo* y la respuesta no debe capturar contenido. Liberamos el framework estilo Buchanan como una contribución metodológica en sí misma.
6. **Manual del defensor y lectura del atacante.** Dos secciones explícitas que traducen los hallazgos en acción concreta. Firmas de detección, canarios de hook, perfiles de comportamiento, y priorización por ranura para ambos lados.
7. **Reproducible y falsable.** Cada afirmación empírica está anclada a una corrida experimental grabada. Cada afirmación arquitectural es rederivable desde el binario público mediante el `discoverer` que liberamos. El sprint de falsificación que precedió a la publicación está documentado en su totalidad.

Hoja de ruta del paper

El paper procede de la siguiente forma. La Sección 2 sitúa este trabajo dentro de dos décadas de ingeniería inversa previa de DRM comercial y los precedentes legales que la han moldeado. La Sección 3 introduce los niveles de licencia de Widevine, el camino de despacho EME, la arquitectura Mojo IPC que usa Chromium para hospedar el CDM, el límite PMP de Windows, el formato Common Encryption (CENC) que usa todo el contenido Widevine, y la estructura de la caja PSSH que transporta datos de inicialización atacante-controlables. La Sección 4 declara el modelo de amenaza y el alcance ético. La Sección 5 recorre la ingeniería inversa estática de CDM_11 en detalle, con listings de disassembly concretos. La Sección 6 describe la instrumentación en runtime, incluyendo las sutilezas de diseño del bypass de CIG, el registro de bitmap CFG, y la cadena de back-pointers por ranura. La Sección 7 presenta los hallazgos empíricos. La Sección 8 trata la distribución de ofuscación como su propia defensa arquitectural. La Sección 9 establece la estabilidad inter-binarios e inter-fuentes. La Sección 10 reporta sobre las pruebas de falsificación, incluyendo los hallazgos candidatos que fueron refutados por análisis más profundo. La Sección ?? discute lo que el modelo implica tanto para atacantes como defensores del DRM de escritorio. La Sección 14 declara qué no se hizo y por qué. La Sección 15 concluye.

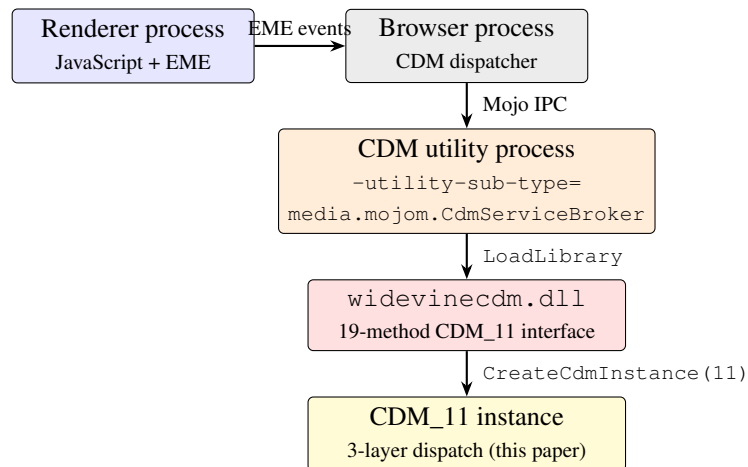


Figura 1: Despacho Widevine EME extremo-a-extremo en navegadores de la familia Chromium sobre Windows. El navegador anfitrión delega las operaciones de Encrypted Media Extensions vía Mojo IPC a un proceso utility cuyo único propósito es cargar `widevinecdm.dll` y exponer su interfaz CDM_11. Este paper estudia el modelo de despacho que se ubica entre la interfaz devuelta por `CreateCdmInstance` y los handlers por-ranura dentro de la biblioteca.

2. Antecedentes y trabajos relacionados

La ingeniería inversa de DRM ocupa un rincón inusual de la investigación en seguridad: abarca criptografía aplicada, ingeniería inversa de binarios, derecho de copyright, relaciones con vendors, y (a veces) prosecución penal. La metodología que seguimos en este paper no surgió en el vacío; está moldeada por dos décadas de trabajo previo, parte del cual produjo resultados técnicos excelentes y parte del cual produjo resultados técnicos excelentes y consecuencias legales adversas para el investigador. Esta sección recorre el estado del arte previo y explica qué precedentes informaron las decisiones metodológicas tomadas aquí.

2.1. El modelo Buchanan

El marco metodológico que seguimos está más claramente articulado en el trabajo reciente de David Buchanan [10, 11, 12]. La investigación de Buchanan incluye notablemente la ingeniería inversa del envoltorio de clave basado en NaCl de iMessage de Apple, la cadena de arranque seguro del chip de seguridad T2 de Apple, y un bug arbitrario en imágenes en la cadena del decoder de imágenes de iOS (CVE-2023-4863).

La propiedad definitoria del trabajo publicado de Buchanan, para nuestros fines, es la separación estricta entre *descripción arquitectural* y *artefacto explotable*. Su output público documenta cómo está estructurado un sistema y dónde están sus fronteras de confianza; las pruebas de concepto controladas que acompañan los writeups demuestran la clase de bug sin producir una herramienta que directamente habilite el abuso contra víctimas no relacionadas. Él describió públicamente la disciplina de publicación resultante como: explorar todo, documentar la técnica, publicar la arquitectura, no publicar el artefacto que permitiría a cualquier lector materializar el impacto.

Para investigación de DRM esta distinción importa: el 17 U.S.C. §1201 (la provisión anti-circumvención del DMCA) y sus equivalentes internacionales prohíben la publicación de herramientas o tecnologías cuyo uso *principal* sea la circumvención de una medida tecnológica de protección. Una descripción arquitectural de un CDM no es esa clase de herramienta. Un frame ripper que consume la misma descripción arquitectural y produce MP4s de Netflix descifrados claramente sí lo es. El espacio entre ambos es la línea que cuidadosamente no cruzamos.

2.2. Hacking the Xbox y la tradición bunnie

El trabajo de Andrew “bunnie” Huang sobre ingeniería inversa de consolas — la Xbox original [13], el pipeline de video con manejo de HDCP del proyecto NeTV [14], y la laptop Novena — demostró un modelo sustentable de investigación adyacente al DRM que era simultáneamente útil, publicable, y no objeto de litigación por parte del vendor. El proyecto NeTV en particular es significativo: HDCP es una medida de protección de copyright bajo la ley estadounidense, y el proyecto requirió correspondencia coordinada con la EFF dirigida a la Library of Congress para confirmar que una demostración arquitectural que no captura, redistribuye, ni facilita la redistribución de contenido protegido se mantiene legal bajo el marco de excepciones del DMCA.

La observación recurrente de Huang es que la exposición legal se adhiere a la *exhibición* de una capacidad de circumvención y a la *distribución* de herramientas que la realizan, no al acto de caracterizar un sistema. El modelo de despacho CDM_11 descrito en este paper sería obviamente útil para un atacante que ya poseyera una capacidad independiente de recibir contenido Widevine cifrado y obtener licencias válidas; sería

esencialmente inútil para un atacante casual que no la tuviera. Esa distribución de utilidad está, en nuestra opinión, del lado correcto de la línea bunnie.

2.3. Sklyarov, DVD-Jon, y la frontera legal

Los casos de **Dimitry Sklyarov** y **Jon Lech Johansen** delimitan la frontera legal de la investigación de DRM.

Sklyarov, un desarrollador ruso de ElcomSoft, fue arrestado en 2001 durante DEF CON por su trabajo en el Advanced eBook Processor, que podía convertir los eBooks protegidos de Adobe en PDF [15]. La prosecución procedió contra ElcomSoft (Sklyarov fue eventualmente liberado tras aceptar testificar). El arresto, el juicio, y la eventual absolución de ElcomSoft sobre los cargos penales establecieron dos precedentes relevantes: (i) los fiscales van a tratar a una herramienta cuya salida es la forma en texto plano de una obra protegida como “usada primariamente para circunvenir”, aunque la investigación subyacente sea académicamente defendible; (ii) meramente *describir* la técnica de circumvención, sin distribuir una herramienta que la realice, no fue la base de la prosecución.

Jon Lech Johansen, “DVD-Jon”, fue procesado por las autoridades noruegas (absuelto dos veces, finalmente definitivamente) por su rol en el desarrollo de DeCSS, la herramienta que rompió el scrambling CSS en los DVDs [16]. Los tribunales noruegos sostuvieron que el trabajo de Johansen sobre su *propio* reproductor de DVD era legal, aunque la herramienta resultante haya sido redistribuida ampliamente en internet. El caso estadounidense de *Universal v. Reimerdes* sostuvo que la *publicación* de DeCSS en un sitio web era circumvención accionable del DMCA, sin importar la legalidad de la investigación subyacente [17].

La lección compuesta de estos casos informa nuestra disciplina de publicación: una descripción arquitectural no es una herramienta; una herramienta que materializa contenido descifrado sí es una herramienta. Publicamos la descripción arquitectural y el marco de validación controlada que prueba que la descripción es correcta; no publicamos una herramienta.

2.4. Trabajo académico sobre Widevine

El ecosistema Widevine ha recibido atención desde varios grupos académicos y corporativos, principalmente enfocados en la variante Android L3.

Quarkslab publicó una serie de análisis de Widevine en Android beginning in 2018 [19, 20]. Their work characterised the OEMCrypto-protected components of the Widevine SDK and demonstrated that the L3 keybox-based protection model relied substantially on software obfuscation that could be peeled back with whitebox-crypto técnicas de ataque. Los proyectos del L3 decryptor que siguieron [21, 22] — la mayoría publicados como open-source de calidad investigación — demostraron extracción extremo-a-extremo de contenido Widevine L3 desde la biblioteca protegida. La respuesta de Google fue deprecarse el soporte de L3 en navegadores web y acelerar la migración a L1.

Check Point Research ha analizado periódicamente Widevine sobre handsets Android, identificando vulnerabilidades específicas de extracción de keybox y clases de bugs en la TrustZone de OEMCrypto [23]. Sus reportes fueron coordinados con Google y el vendor antes de su publicación.

Romain Thomas y colaboradores de Quarkslab han publicado extensamente sobre herramientas prácticas de ingeniería inversa que usamos aquí, incluyendo la biblioteca LIEF para parsing multiplataforma de binarios [24] y varios posts de blog sobre whitebox crypto en DRM comercial. Su trabajo en herramientas está directamente upstream de la metodología usada en este paper para análisis estático de `widevinecdm.dll`.

No existe, hasta donde sabemos, ningún estudio académico publicado sobre los internals de despacho del CDM L1 de Windows como tal. El trabajo más cercano en espíritu es un puñado de posts de blog y threads de foros que documentan issues específicos (por ejemplo, el layout de símbolos de `libwidevinecdm.so` en Linux, los cambios de ABI del CDM entre versiones mayores) sin proponer un modelo arquitectural generalizable. Esa es la brecha que el presente paper intenta llenar para el build L1 de Windows.

2.5. Criptografía whitebox y ofuscación MBA

Los patrones de ofuscación observados en los handlers de capa-3 de CDM_11 — constantes mágicas de alta entropía escritas al stack en la entrada de la función, instrucciones de avance de estado por hash multiplicativo, cascadas `cmovae` sin branches, rellenos de poison `0xaaaaaaaa` — coinciden con la firma de libro de texto de la ofuscación *mixed-boolean-arithmetic* (MBA) tal como se trata en la literatura académica [25, 26]. La ofuscación MBA transforma una operación aritmética en una expresión equivalente que mezcla operaciones bit-a-bit y aritméticas de modo que derrota la simplificación algebraica sobre la forma ofuscada. Se usa ampliamente en productos comerciales de protección de software (VMProtect, Themida, Arxan) y en ofusadores a medida contruidos por vendors para casos defensivos específicos.

Una técnica distinta pero relacionada, la *criptografía whitebox*, embebe material de clave criptográfica en representaciones basadas en tablas que resisten la extracción directa incluso cuando el atacante tiene acceso completo de lectura al código ofuscado [27, 28]. Los productos comerciales de DRM usan variantes de whitebox-AES extensivamente; las claves cifradas en una licencia Widevine L3 son desenvueltas por exactamente esa clase de rutina antes de ser aplicadas al descifrado por-cuadro.

El patrón que reportamos en la Sección 8 — ofuscación concentrada a la profundidad donde los datos estructurados del atacante empiezan a ser parseados — es consistente con la prescripción de diseño de la literatura whitebox: proteger al procesador criptográfico, no el pegamento de despacho a su alrededor. Si la protección *tiene éxito* contra un atacante determinado es una pregunta separada que no abordamos.

2.6. Antecedentes de herramientas

Tres piezas concretas de herramientas en las que se apoya este trabajo merecen reconocimiento:

Capstone [29], el framework de disassembly multi-arquitectura usado por cada script en este release que recorre la sección `.text` de `widevinecdm.dll`.

pefile [30], el parser PE puro de Python usado para enumerar exports, recorrer la tabla de secciones, y leer RVAs desde `.rdata`.

mingw-w64, la cross-toolchain de Windows usada para construir el shim y los helpers desde un host Linux. Notablemente, el backend de gcc de mingw-w64 no soporta el structured exception handling `__try` / `__except` de Microsoft, lo que nos forzó a usar checks de seguridad basados en `VirtualQuery` para los caminos de lectura del shim que usan registers como target, en lugar de `memcpy` protegido por SEH. Esta es la clase de pequeño detalle de implementación que no aparece en la literatura llamativa de investigación de DRM pero que determina si la metodología corre o no.

3. Marco técnico

Esta sección revisa los componentes técnicos principales del pipeline de reproducción Widevine L1 de Windows en mayor profundidad de la que permitió la introducción.

3.1. Niveles de licencia de Widevine

Widevine define tres niveles de seguridad anidados.

L3 es un CDM puramente software. El estado criptográfico se mantiene en memoria del proceso dentro de un runtime ofuscado. Las claves de contenido se desenvuelven mediante un núcleo criptográfico estilo whitebox y se aplican a segmentos de ciphertext en buffers user-space. La salida descifrada se devuelve al anfitrión a través de memoria estándar. Según la política de licencia Widevine publicada, los clientes solo-L3 tienen techo en 540p; los acuerdos de los dueños de contenido frecuentemente restringen L3 a tasas de bits más bajas que las que reciben los clientes L2/L1.

L2 es híbrido: un CDM software mantiene al menos sus operaciones sensibles detrás de un Trusted Execution Environment (TEE) respaldado por hardware en el anfitrión. L2 ve un deployment limitado en escritorio.

L1, la variante estudiada en este paper, requiere que el anfitrión provea un pipeline de medios protegido respaldado por hardware. En Windows el pipeline relevante es el Protected Media Path (PMP), él mismo un compuesto de:

- `peauth.sys`, the Microsoft-signed kernel-mode component that enforces the OPM contract between the CDM and the GPU driver,
- the Media Foundation `IDirectXVideoDecoder` chain running in a kernel-protected mode,
- the DXGI protected-surface allocator that exposes the decoded frame only to the GPU presentation pipeline and not to user-space readback.

Under L1 hardware decode, the decrypted frame never appears in user-mode browser process memory. This is the design property that the paper's first empirical finding (Section 7.1) exploits to demonstrate the CDM dormancy phenomenon.

3.2. Encrypted Media Extensions and the dispatch path

La especificación W3C Encrypted Media Extensions [7] define la API de JavaScript por la cual una aplicación web solicita reproducción mediada por DRM de un stream de medios cifrado. Desde el punto de vista del navegador anfitrión, las operaciones relevantes son:

1. `navigator.requestMediaKeySystemAccess("com.widevine.alpha")`: confirma que el cliente tiene un CDM para el key system nombrado.
2. Crear un objeto `MediaKeys` y vincularlo a un `HTMLMediaElement`.
3. Esperar el evento `encrypted`, que provee los datos de inicialización Common Encryption (CENC) extraídos del stream de medios.
4. Crear una `MediaKeySession` y llamar a `generateRequest(initDataType, initData)`, lo que causa que el CDM construya un request de licencia.

5. Reenviar el mensaje resultante al servidor de licencias, recibir la respuesta, y devolverla vía `updateSession(licenseResponse)`.
6. Continuar la reproducción mientras el CDM desenvuelve las claves de contenido y las aplica a los datos del medio cifrado.

La implementación de Chromium de esta API enruta cada paso a través del proceso navegador, el renderer, y el proceso utility del CDM vía interfaces Mojo IPC definidas en `chromium/components/cdm` [3]. La interfaz Mojo relevante para nuestros fines es `media.mojom.ContentDecryptionModule`, cuyos mensajes de request y response son esencialmente una proyección serializada de la interfaz C++ de diecinueve métodos de `CDM_11`.

3.3. La interfaz C++ de CDM_11

La interfaz que `CDM_11` implementa está definida en el árbol de Chromium en `media/cdm/api/content_decryption_module.h`. La clase es una base puramente virtual `ContentDecryptionModule_11` con las siguientes diecinueve entradas, declaradas en este orden:

```

1 virtual void Initialize(...) = 0;
2 virtual void GetStatusForPolicy(...) = 0;
3 virtual void SetServerCertificate(...) = 0;
4 virtual void CreateSessionAndGenerateRequest(...) = 0;
5 virtual void LoadSession(...) = 0;
6 virtual void UpdateSession(...) = 0;
7 virtual void CloseSession(...) = 0;
8 virtual void RemoveSession(...) = 0;
9 virtual void TimerExpired(...) = 0;
10 virtual Status Decrypt(...) = 0;
11 virtual Status InitializeAudioDecoder(...) = 0;
12 virtual Status InitializeVideoDecoder(...) = 0;
13 virtual void DeinitializeDecoder(...) = 0;
14 virtual void ResetDecoder(...) = 0;
15 virtual Status DecryptAndDecodeFrame(...) = 0;
16 virtual Status DecryptAndDecodeSamples(...) = 0;
17 virtual void OnPlatformChallengeResponse(...) = 0;
18 virtual void OnQueryOutputProtectionStatus(...) = 0;
19 virtual void OnStorageId(...) = 0;

```

Cada llamada virtual se traduce, bajo la convención de llamada x64 de Microsoft, en una única llamada indirecta a través de un function pointer cargado desde la vtable del objeto. El layout de la vtable para una clase con diecinueve funciones virtuales y sin virtuales de clase base es un arreglo contiguo de diecinueve punteros a código; el overhead por llamada desde la perspectiva del anfitrión es una carga indirecta más el costo del dispatcher CFG.

Adoptamos la convención de identificar los métodos `CDM_11` por su índice de ranura a lo largo de este paper. `Decrypt` es la ranura 9, `DecryptAndDecodeFrame` es la ranura 14, y así.

3.4. Common Encryption (CENC) y PSSH

El protocolo Widevine transporta sus datos de inicialización dentro de una caja Protection-Specific System Header (PSSH), tal como está estandarizada en ISO/IEC 23001-7 (Common Encryption para ISO Base Media File Format) [9, 8]. Una caja PSSH está estructurada así:

```

1 | PSSHBox {

```

```

2   u32      size;           // big-endian
3   char[4]  type;          // "pssh"
4   u8       version;
5   u24      flags;
6   UUID(16B) SystemID;     // Widevine SystemID
7                               // edef8ba9-79d6-4ace-a3c8-27dcd51d21ed
8
9   if version > 0:
10      u32      KID_count;
11      UUID[KID_count] KIDs;
12      u32      data_size;
13      u8[data_size] data;
14 }

```

El campo `data`, para Widevine, contiene una serialización Protocol Buffers de un mensaje `WidevineCencHeader` con un esquema conocido. La llamada `generateRequest` pasa la caja PSSH completa al CDM, que parsea el `SystemID`, los `KIDs`, y el header codificado en protobuf para construir un request de licencia.

Este es el input estructurado atacante-controlable que ingiere la ranura `CreateSessionAndGenerateRequest` (ranura 3): un manifest DASH malicioso o una aplicación EME maliciosa puede colocar bytes arbitrarios dentro del PSSH y pasarlos al CDM con control total tanto sobre los campos del header de la caja como del payload protobuf. Las vulnerabilidades de parser en el camino PSSH del CDM son alcanzables por atacantes desde cualquier CDN comprometido que sirve el manifest DASH, desde cualquier JavaScript comprometido que conduce la aplicación EME, o desde cualquier atacante MITM que pueda reescribir el manifest DASH en tránsito.

Los dos esquemas estándar de cifrado CENC que el CDM debe soportar son **CENC-CTR** (AES-128 en modo Counter, con descriptores de subsample que indican qué rangos de bytes del track cifrado están protegidos) y **CBCS** (AES-128 en modo CBC con cifrado por *patrón*: un patrón 1-de-10 o 1-de-9 de bloques cifrados-salteados diseñado para tolerar los límites de NALU de AVC/HEVC). La estructura `InputBuffer_2` de `CDM_11` transporta ambos esquemas y sus parámetros de subsample/patrón por llamada.

3.5. Mojo IPC y el proceso utility del CDM

Chromium aísla al CDM en un proceso utility dedicado para confinar el impacto del compromiso del CDM. El proceso utility se spawnea con la línea de comando:

```

1 chrome.exe --type=utility
2   --utility-sub-type=media.mojom.CdmServiceBroker
3   --lang=en-US --service-sandbox-type=cdm
4   --no-sandbox (when CIG is disabled for research)
5   ...

```

Dentro del utility, una implementación de interfaz Mojo acepta los requests CDM del anfitrión y los reenvía a una clase C++ que posee un único puntero a `ContentDecryptionModule_11`. Ese puntero apunta al objeto devuelto por `widevinecdm.dll!CreateCdmInstance(11, ...)` — que es el objeto cuya vtable mapeamos en la Sección 5.

3.6. Control Flow Guard y Code Integrity Guard

Dos mitigaciones a nivel de proceso de Windows son relevantes para la instrumentación en runtime del utility del CDM:

Code Integrity Guard (CIG) es la política de mitigación de proceso `PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON`. Cuando está habilitada, el kernel rechaza cargar cualquier DLL al proceso que no esté firmado por Microsoft o un partner de Microsoft. Chrome habilita CIG en sus procesos `renderer` y `utility` por default desde Chrome 70 [4]. La flag `-disable-features=RendererCodeIntegrity`, en combinación con `-no-sandbox` en los procesos afectados, deshabilita CIG para uso de investigación.

Control Flow Guard (CFG) es una mitigación a nivel de compilador de Microsoft que agrega un bitmap validado por kernel de targets válidos de indirect-call al proceso corriendo e inserta un check antes de cada indirect call [5]. Una vtable parcheada que apunta a una función hook que recién asignamos no está en el bitmap por default, así que la primera llamada por la ranura parcheada dispara un fallo CFG que termina el proceso.

La forma soportada de marcar una nueva función como target válido es la API `SetProcessValidCallTargets` [6]. La API toma un handle de proceso, una base de página alineada a 4 KB, un tamaño de página, y un array de descriptores `CFG_CALL_TARGET_INFO` cuyo campo `Offset` es el *offset intra-página* (12 bits bajos) del target. Una versión temprana de nuestro inyector usó los 16 bits bajos, lo cual está mal: la página que respalda al hook es de 4 KB y el bitmap se indexa con granularidad de 16 bytes dentro de la página. El inyector liberado usa la máscara correcta:

```
1 CFG_CALL_TARGET_INFO cti;
2 cti.Offset = (DWORD)(hookVA & 0xfff); /* in-page offset */
3 cti.Flags = CFG_CALL_TARGET_VALID;
4 DWORD_PTR pageBase = hookVA & ~0xfffULL;
5 SetProcessValidCallTargets(hProc, (PVOID)pageBase,
6                             0x1000, 1, &cti);
```

3.7. ASLR y la vista compartida del DLL

Un detalle práctico de implementación relevante a nuestro experimento entre pestañas (Sección 7.4): los procesos `utility` de Chrome cargan `widevinecdm.dll` desde la misma imagen en disco cada vez, y la compartición de `image-sections` de Windows significa que dos procesos `utility` separados ambos terminan con el DLL mapeado en la *misma dirección virtual*. Por eso las direcciones de hook de nuestro inyector (en la imagen del shim, en la base de carga reservada del shim) fueron idénticas entre el proceso `utility` del CDM de Netflix y el de Bitmovin. Sin embargo, la sección de datos del DLL es `copy-on-write` por proceso; cada `utility` tiene sus propios contadores y su propio file handle de log abierto.

4. Modelo de amenaza y metodología

4.1. Modelo de amenaza

Consideramos a un usuario honesto de un navegador de la familia Chromium reproduciendo contenido Widevine-protegido con decodificación por hardware en Windows 11. El usuario ha consentido a la instrumentación nivel-investigación de su propio navegador y ha instalado nuestras herramientas de ingeniería inversa en su propia máquina de laboratorio. El ejecutable del navegador no está modificado y está firmado por el vendor. El CDM Widevine es el `widevinecdm.dll` provisto por el vendor de la versión del navegador instalado. El usuario posee una suscripción válida y paga al servicio comercial de streaming usado en las secciones empíricas del paper, y el uso de esa suscripción es para visualización personal durante la ventana de investigación. No hay segundo usuario, ni jurisdicción fuera de la propia del usuario, ni contenido de terceros involucrado.

Excluimos deliberadamente de este estudio cualquier modelo de amenaza en el que un atacante capture cuadros descifrados. Nuestra preocupación no es el potencial de explotación del CDM sino su forma arquitectural y la superficie de ataque que presenta al navegador anfitrión y al atacante de red posicionado entre el navegador y el CDN del streaming.

4.2. Invariantes de la metodología

Seguimos lo que describimos como el *modelo Buchanan* de investigación de DRM, resumido anteriormente (Sección 2.1). La versión operativa del modelo que gobernó cada decisión experimental en este paper es:

1. **Observación shape-only.** A cualquier hook runtime se le permite registrar metadata de despacho — número de ranura, timestamp, hashes de valores de registros, campos de tamaño de estructura — pero se le prohíbe copiar cualquier byte apuntado por puntero perteneciente a ciphertext, cuadros descifrados, material de clave de contenido, o vectores de inicialización, hacia cualquier almacenamiento persistente o cualquier canal fuera del proceso. El código fuente del shim codifica esta invariante explícitamente: no existe camino de código que llame a `memcpy(&log_buf, ib.data, ...)`; el extractor de forma sólo lee los campos `size`, un hash FNV-1a-32 de los primeros 16 bytes de `key_id`, una flag de presencia distinta-de-cero para el IV, y un puñado de campos escalares públicos.
2. **Ausencia demostrable.** Después de cada experimento verificamos forensicamente que el almacenamiento producido es JSON puro ASCII consistente sólo de los campos de metadata permitidos y no contiene blobs binarios, ni contenido decodificado, ni material de clave. Documentamos el procedimiento de verificación en la Sección 14 y proveemos el script verificador en la metodología liberada.
3. **Mutación reversible.** Todas las modificaciones runtime al proceso target (parches de vtable, instalaciones de hook, registros de bitmap CFG) son reversibles. Guardamos el estado pre-parche en formato wire al tooling liberado y restauramos el target a su estado original antes de publicar. El helper liberado retiene las direcciones originales de los thunks de la outer-vtable en el array `g_pOrigSlot[19]` del shim para que un observador externo pueda validar que el estado post-experimento es byte-igual al estado pre-experimento.

4. **Sin superficie de circumvención de contenido.** Usamos la propia sesión de navegador del operador a un servicio comercial de streaming para confirmar qué hace el CDM durante steady-state, pero nunca instrumentamos al decoder, ni al componente PMP del lado kernel (`peauth.sys`), ni la superficie protegida, ni cualquier otro componente posicionado para materializar un cuadro descifrado. El shim shape-only es el hook más profundo en el stack que instalamos.
5. **Falsificación antes de la publicación.** Cada afirmación empírica se somete a una prueba de falsificación: una hipótesis alternativa que, si fuera verdadera, invalidaría la afirmación, se construye y se prueba con cualquier evidencia que la metodología pueda producir. La Sección 10 reporta el sprint de falsificación.

Una consecuencia en lenguaje natural de estas invariantes es que nuestros resultados caracterizan el *comportamiento de despacho* del CDM, no su *estado criptográfico*. Podemos reportar con qué frecuencia dispara una ranura durante reproducción real, qué distribución de tuplas de registros ve, cuál es la forma de su descriptor de buffer entrante. No podemos y no reportamos ningún byte de contenido decodificado.

5. Ingeniería inversa estática de CDM_11

Esta sección recorre la ingeniería inversa estática del despacho CDM_11 en `widevinecdm.dll`, en el orden en que la realizamos. Dos binarios fueron analizados en paralelo: **Chrome 149.0.7827.115**, desde su path de instalación estándar bajo `WidevineCdm/_platform_specific/win_x64/` (~22 MB), y la misma biblioteca tal como viene en **Microsoft Edge 150** (~19 MB). Todos los RVAs se miden respecto al image base del binario.

5.1. Paso 1: `CreateCdmInstance` y el switch de versión

La tabla de exports de `widevinecdm.dll` contiene seis entradas; la única relevante para nuestros propósitos es `CreateCdmInstance`:

```
1 Edge 150      CreateCdmInstance @ rva 0x1d0850
2 Chrome 149    CreateCdmInstance @ rva 0x1d2e10
```

Disasemblando cualquiera de las dos entradas muestra una cadena de pequeñas funciones wrapper que cargan una palabra de versión en `ecx` (versión 9, 10, u 11 de la interfaz CDM) y llaman a un constructor por-versión. El disassembly del fragmento relevante del `CreateCdmInstance` de Edge 150 es:

```
1 0x1801d0850: push    r14
2 0x1801d0852: push    rsi
3 0x1801d0853: push    rdi
4 0x1801d0854: sub     rsp, 0x40
5 0x1801d0858: mov     edi, ecx          ; ecx = requested version
6 0x1801d085a: lea     rax, [rip + 0x...] ; load module-relative base
7 ...
8 0x1801d0994: mov     ecx, 0xb          ; CDM_11 selected
9 0x1801d0999: mov     rax, r9           ; arg setup
10 ...
```

Escanear ingenuamente el primer `call` después de cada `mov ecx, 0xb` devuelve el stack-check probe del runtime (`__chkstk_ms` o equivalente), no el constructor real. Un discoverer robusto debe filtrar helpers conocidos del runtime. El script liberado `cdm_arch_discover.py` usa una heurística estructural: enumera todos los targets de `call` alcanzables desde `CreateCdmInstance`, filtra targets llamados más de dos veces (el constructor se llama exactamente una vez por branch de versión; los helpers de runtime se llaman muchas veces), y valida cada candidato inspeccionando si su cuerpo contiene una secuencia de la forma:

```
1 lea reg, [rip + N]      ; load a candidate vtable RVA
2 mov qword ptr [r?], reg ; install at offset 0 of 'this'
```

y si el RVA cargado apunta dentro de `.rdata` a una ubicación cuyos primeros diecinueve QWORDS son todos punteros de código válidos dentro de `.text`. El primer candidato que cumpla se reporta como el constructor del CDM y el RVA cargado como su outer vtable. El discoverer localiza el RVA de outer vtable en Edge 150 en `0x780250` por este camino; el outer vtable de Chrome 149 en `0x768fa0`.

5.2. Paso 2: Versiones co-empaquetadas del CDM

Un escaneo *exhaustivo* posterior de `.rdata` en busca de estructuras de diecinueve ranuras que coincidan con el patrón thunk del CDM (descrito abajo) revela **dos** de esas estructuras por binario, espaciadas a `0xc0` bytes:

Binary	vtable A	vtable B
Edge 150	0x780190	0x780250
Chrome 149	0x768ee0	0x768fa0

Dentro de cada binario, ambas vttables comparten su entrada de descifrado por-cuadro (la ranura 14 apunta a la misma función tanto en A como en B), pero sus entradas `Initialize` difieren:

Binary	vtable A.slot[0]	vtable B.slot[0]
Edge 150	0x1db330	0x1db390
Chrome 149	0x1e03c0	0x1e0420

Esto es consistente con que el binario empaqueta tanto vttables CDM_10 como CDM_11 en soporte a navegadores anfitriones que aún no se han actualizado a la interfaz más reciente, y coincide con el historial del header de Chromium. En lo que sigue identificamos a la *vtable B* como la outer vtable de CDM_11, dado que parches en runtime en las posiciones de ranuras de *vtable B* interceptan el despacho frente al anfitrión EME (Sección 6). La implementación compartida en la ranura 14 refleja la decisión de implementación de usar una rutina de descifrado por-cuadro a lo largo de ambas versiones de interfaz; la ranura 0 divergente refleja la firma `Initialize` específica de versión.

5.3. Paso 3: El patrón thunk de la vtable externa

La outer vtable contiene diecinueve entradas. La mayoría de ellas son *thunks* de la forma:

```

1 slot N thunk @ rva 0x1dad00 + (N-3)*0x20:
2   mov rcx, qword ptr [rcx + 8]      ; this->m_impl
3   mov rax, qword ptr [rcx]         ; m_impl->m_vtable
4   mov rax, qword ptr [rax + 8*N]    ; slot N of m_impl_vtable
5   mov rdx, qword ptr [rip + cfg_disp]; CFG dispatcher pointer
6   jmp rdx                          ; CFG-protected tail-call

```

Cada thunk son veintiocho bytes seguidos de relleno `int3`, dispuestos en una secuencia uniforme con espaciado `0x20` dentro de `.text` para las ranuras 3 a 17. El thunk de la ranura 14 en Edge 150, disassembled por completo:

```

1 0x1801dae60: mov    rcx, qword ptr [rcx + 8]
2 0x1801dae64: mov    rax, qword ptr [rcx]
3 0x1801dae67: mov    rax, qword ptr [rax + 0x70]
4 0x1801dae6b: mov    rdx, qword ptr [rip + 0x10dcd2e]
5 0x1801dae72: jmp    rdx
6 0x1801dae74: int3
7 0x1801dae75: int3
8 ... (int3 padding to 0x20 boundary)

```

Disassembamos cada thunk en ambos binarios y confirmamos el chequeo de índice de ranura (el offset en `[rax + 8*N]` es igual al número de ranura multiplicado por ocho) para quince de quince ranuras thunkeadas en cada binario. La confirmación byte-a-byte es lo que licencia nuestra afirmación de “inactividad de la ranura 14” en la Sección 7.1: la evidencia en runtime ($g_callsBySlot[14] = 0$) es atribuible al método correcto sólo porque el índice de ranura en el thunk de la outer-vtable despacha al offset interno correcto.

Las ranuras 0 (`Initialize`), 1 (`GetStatusForPolicy`) y 18 (`OnStorageId`) *no* son thunks; son implementaciones directas ubicadas en `.text` en direcciones no contiguas. La ranura 2

(SetServerCertificate) es un thunk en ambos binarios pero despacha vía `[rax + 8]` en lugar de `[rax + 0x10]` — comparte una implementación con la ranura 1 a nivel de la capa `m_impl`. Este es un detalle arquitectural por-binario consistente tanto en Edge 150 como en Chrome 149, sugiriendo que es una decisión deliberada de implementación más que un accidente del build.

5.4. Paso 4: La vtable `m_impl`

El thunk externo despacha hacia una vtable *intermedia* poseída por el objeto `m_impl` que la clase exterior mantiene en `this->m_impl([outer + 8])`. Localizar esta vtable intermedia en `.rdata` es el segundo paso del análisis estático.

Usamos un pivote de firma conocida. De una traza de call-chain previa (documentada en nuestras notas de investigación), sabíamos que el wrapper interno de la ranura 5 (`UpdateSession`) en Edge 150 está en RVA `0x1d4ab0`, identificable por su prólogo multi-push con un stack canary que carga datos atacante-controlables a través de una copia camino al parser ofuscado. El código del wrapper es inconfundible en un listing de disassembly.

Para encontrar la vtable `m_impl`, hacemos una búsqueda reversa en `.rdata` del valor `QWORD image_base + 0x1d4ab0`. Hay exactamente una ocurrencia. Retroceder cinco entradas de ocho bytes desde el match da la dirección de la vtable `m_impl`: RVA `0x77d370` en Edge 150. Los diecinueve `QWORDS` circundantes apuntan dentro de `.text`, confirmando al candidato. El mismo pivote aplicado a Chrome 149 (donde el wrapper interno de la ranura 5 está en el RVA análogo) localiza la vtable `m_impl` de Chrome 149 en el offset correspondiente.

5.5. Paso 5: La estructura de la capa `m_impl`

Disasemblando cada entrada de la vtable `m_impl` se revela una mezcla de tres tipos de cuerpo:

1. **Wrappers MSVC limpios** (ranuras 2, 3, 4, 5, 6, 7, 18). Tienen un prólogo estándar (múltiples `push` callees, `sub rsp, N`, carga del stack canary), allocan o piden prestadas copias de input, realizan saneamiento, y luego llaman a un handler más profundo. Estas son las ranuras que ingieren datos de longitud variable influenciados por el atacante (strings de session-id, payloads de respuesta, buffers de init-data). El wrapper de la ranura 3 en Edge 150, abreviado para el paper:

```

1 0x1801d47d0: push    r15
2 0x1801d47d2: push    r14
3 0x1801d47d4: push    r13
4 0x1801d47d6: push    r12
5 0x1801d47d8: push    rsi
6 0x1801d47d9: push    rdi
7 0x1801d47da: push    rbx
8 0x1801d47db: sub     rsp, 0x50
9 0x1801d47df: mov     esi, r9d           ; init_data_type
10 0x1801d47e2: mov     edi, r8d           ; session_type
11 0x1801d47e5: mov     ebx, edx           ; promise_id
12 0x1801d47e7: mov     eax, [rsp + 0xb8]   ; init_data_size
13 0x1801d47ee: mov     rdx, [rip + 0x10f284b]
14 0x1801d47f5: xor     rdx, rsp           ; stack canary
15 0x1801d47f8: mov     [rsp + 0x48], rdx
16 0x1801d47fd: mov     r14, [rcx + 0x10]   ; load m_impl-internal
17 0x1801d4801: xorps   xmm0, xmm0
18 0x1801d4804: movaps  [rsp + 0x30], xmm0
19 ...
20 0x1801d4821: mov     rcx, r12

```

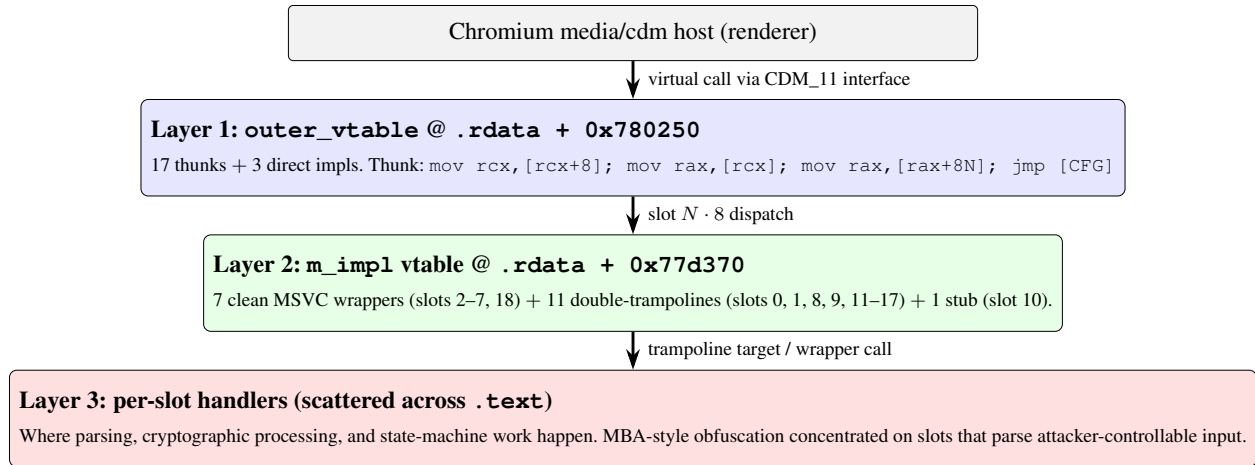


Figura 2: El modelo de despacho de tres capas de CDM_11. Cada llamada virtual pública del CDM desde el anfitrión atraviesa tres indirecciones antes de alcanzar la función que realiza el trabajo. Los RVAs mostrados son de Edge 150 `widevinecdm.dll`; Chrome 149 tiene la misma estructura en RVAs correspondientemente diferentes (ver Sección 9).

```

21 0x1801d4824: call    0x180736080      ; malloc/__chkstk
22 0x1801d4829: mov     [rsp + 0x30], rax
23 0x1801d482e: lea     r13, [rax + r12]      ; end pointer
24 ...
25 0x1801d485f: call    0x18021f200          ; LAYER-3 ENTRY
26                                     ; (obfuscated parser)
  
```

2. **Double-trampolines** (ranuras 0, 1, 8, 9, 11–17). Cuerpos de dos instrucciones de la forma `mov rcx, [rcx + N]` seguido por `jmp <addr>` que reenvían la llamada a un handler de capa más profunda sin procesamiento intermedio. La entrada de la ranura 14 de `m_impl` en Edge 150:

```

1 0x1801d4f10: mov     rcx, qword ptr [rcx + 0x10]
2 0x1801d4f14: jmp     0x180230c20          ; layer-3 handler (obfuscated)
  
```

3. **Stubs** (ranura 10). Un cuerpo de dos instrucciones `mov eax, 3 ; ret` que devuelve un código de error constante sin más trabajo. La ranura 10 es `InitializeAudioDecoder`; en el build L1 de Windows es intencionalmente un no-op porque la decodificación de audio es manejada por el pipeline Media Foundation fuera del CDM.

La vtable `m_impl`, en otras palabras, es mayormente una capa de *ruteo*; el trabajo real sucede una indirección más adelante.

5.6. Paso 6: El modelo de despacho de tres capas

Componiendo los cinco pasos anteriores, la cadena de despacho desde una llamada virtual del anfitrión hasta un handler por-ranura se muestra en la Figura 2. Cada llamada virtual pública atraviesa tres indirecciones antes de alcanzar la función que realiza el trabajo.

5.7. Una nota sobre los falsos arranques

Para el beneficio de investigadores que sigan este trabajo, el análisis estático tal como en realidad lo llevamos a cabo incluyó dos falsos arranques substanciales.

El primer falso arranque fue identificar una tabla de despacho singleton interna en `.data + 0x1588b48` como la “outer vtable de CDM_11”. Esa tabla, de hecho, es una superficie de despacho interna separada de diecinueve ranuras dentro de la biblioteca Widevine que no es la interfaz pública EME. Nuestra conclusión inicial de que “la ranura por-cuadro de la outer vtable de CDM_11 dispara ~ 600 veces por segundo bajo reproducción HW de Netflix en steady-state” era incorrecta: era la ranura 0 del singleton interno la que disparaba 600 veces por segundo, y la ranura por-cuadro real de la outer-vtable estaba en cero. La corrección vino de un pivote de traza de constructor que localizó la outer vtable real vía el export `CreateCdmInstance`, tras lo cual el hallazgo de inactividad pasó a ser correctamente atribuible a la superficie de cara al usuario.

El segundo falso arranque fue la atribución original de ofuscación estilo MBA a la capa `m_impl`. La capa `m_impl` de hecho es una capa de ruteo limpia (según la Sección 5.5); la ofuscación vive en la capa tres. Una enumeración capa-por-capas de las diecinueve ranuras, documentada en nuestras notas de investigación 13b y 13c, localizó la ofuscación en su profundidad real.

Destacamos estos falsos arranques porque son típicos del trabajo de RE estático sobre binarios cerrados con indirecciones internas de despacho. Una lectura en una sola pasada del call graph del binario naturalmente se habría asentado o en la vtable incorrecta o en la capa incorrecta, y el sprint de falsificación de la Sección 10 es lo que atrapó ambos.

6. Instrumentación en runtime

El mapa estático de la sección anterior arroja hipótesis sobre qué ranuras son alcanzadas durante reproducción en steady-state y cuáles no. Para verificar esas hipótesis instrumentamos el proceso utility del CDM en ejecución con un hook shape-only.

6.1. Code Integrity Guard (CIG) y el flag de launch

Chrome habilita Code Integrity Guard (`PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON`) en sus procesos utility desde Chrome 70 [4]. CIG rechaza cualquier `LoadLibrary` de una imagen no firmada por Microsoft a nivel del kernel, por lo que una inyección vainilla `CreateRemoteThread(LoadLibraryA)` de nuestro shim falla en la etapa `LdrLoadDll` con `STATUS_DYNAMIC_CODE_BLOCKED` (`0xC0000605`).

Para uso de investigación en laboratorio, Chrome puede ser lanzado con las flags `-disable-features=RendererCodeIntegrity` y `-no-sandbox`, que en conjunto deshabilitan CIG en los procesos utility. Esto produce un target de investigación controlado sin modificar la biblioteca CDM ni el binario del navegador anfitrión, y corresponde a la asunción equivalente de alcanzabilidad que un atacante con ejecución de código en el renderer tendría que satisfacer por separado (vía, por ejemplo, una arbitrary-write al JIT del renderer o una cadena de process-load que evada CIG).

6.2. Registro del bitmap de Control Flow Guard

CIG-deshabilitado no deshabilita Control Flow Guard (CFG): una vez que el shim está cargado, simplemente parchar una entrada de vtable para que apunte a una función hook no registrada causa que el dispatcher CFG aborte la llamada como una violación de control de flujo. El installer liberado usa `SetProcessValidCallTargets` para marcar cada hook target como un call target válido antes de parchar su ranura. La Figura 3 bosqueja el flujo de instalación.

6.3. Diseño del shim shape-only

El shim, `wv_shim.dll`, exporta diecinueve funciones hook por-ranura `WvHookSlot0` hasta `WvHookSlot18`, cada una con su propio slot de puntero original en un array exportado `g_pOrigSlot[19]`, y un único array contador `g_callsBySlot[19]`. Cada función hook:

1. Atomically increments its slot counter and the aggregate `g_calls`.
2. Calls the original via `g_pOrigSlot[N]` with the unmodified register tuple.
3. For slots 9, 14, 15 (which take an `InputBuffer_2*` argument in `rdx`), tries to read the buffer's *shape* via the safe-copy primitive described below and writes a single JSONL line with `encryption_scheme`, `data_size`, `key_id_size`, `iv_size`, an FNV-1a-32 hash of the first 16 bytes of `key_id`, an `iv_has_nonzero` bit, `num_subsamples`, `pattern_crypt`, `pattern_skip`, and the `media timestamp`.
4. For other slots, writes a `generic` event with an FNV-1a-32 hash of the four argument registers.

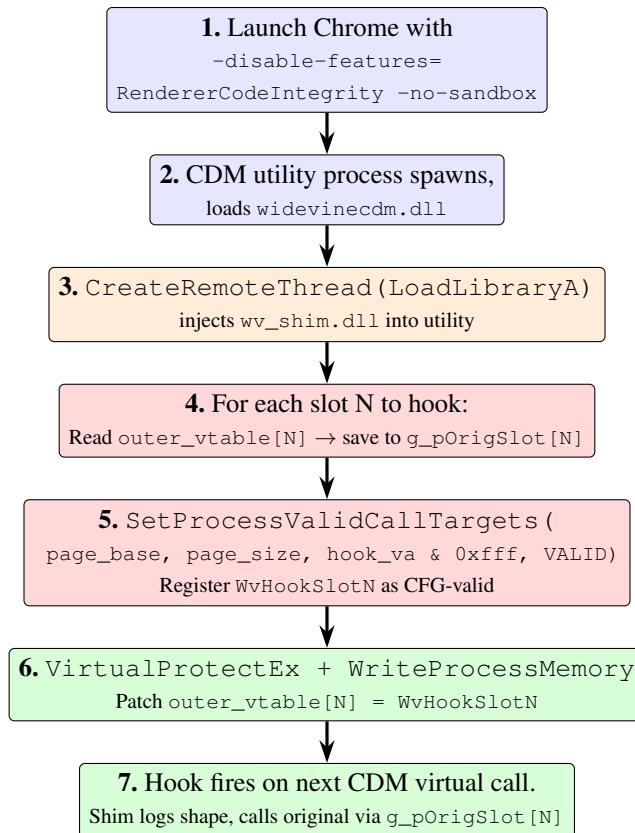


Figura 3: Secuencia de instalación del hook. El Paso 5 (registro del bitmap CFG) es lo que atrapa a los investigadores con más frecuencia — el campo `Offset` de `CFG_CALL_TARGET_INFO` espera un offset intra-página (12 bits bajos) del hook target, no una dirección absoluta ni otra máscara. El Paso 4 captura la sutileza de diseño que motivó al array de back-pointers por-ranura (§6.4).

No byte of `ib.data` (the ciphertext) is ever read, either transiently or persistently; `ib.key_id` and `ib.iv` are read into a stack-local buffer that is collapsed into a hash and then immediately discarded. The shim source enforces this by simply not having a code path that copies any of those fields into a logging structure.

El primitivo `safe-copy` usa `VirtualQuery` para confirmar que cada página fuente está `commit'cada` y es legible antes de copiar, ya que el compilador C de `mingw-w64` no provee el `SEH __try / __except` de Microsoft en `x86_64`. El check recorre cada página del rango fuente, confirma el estado `MEM_COMMIT`, y confirma una de las protecciones de página con permiso de lectura (`PAGE_READONLY`, `PAGE_READWRITE`, `PAGE_EXECUTE_READ`, etc.) antes del `memcpy`.

6.4. Cadena de punteros originales por ranura

Una sutileza de diseño que vale la pena notar: la primera versión del shim usaba una sola variable `g_pOrigDecrypt` para todas las ranuras hookeadas. Parchar múltiples ranuras con ese único back-pointer llevaba a crashes por `signature-mismatch` en el momento que disparaba la segunda ranura parcheada (el back-pointer contenía el original de la ranura anterior). El shim liberado usa un array por-ranura. Aún más sutilmente, cuando hookeamos la *vtable exterior* en lugar de una tabla singleton interna, el back-pointer debe contener la dirección del thunk exterior, *no* una dirección de tabla de despacho interna; si no, el camino de llamada-original termina saltando a la función interna incorrecta y corrompe silenciosamente la llamada.

Nuestro installer (wv_helper3.c) lee el original de la outer vtable en el momento del parche y lo guarda en g_pOrigSlot[N] como parte de la misma operación atómica:

```
1 DWORD_PTR origFn = 0;
2 ReadProcessMemory(hp, (LPCVOID)slotAddr, &origFn, 8, NULL);
3
4 /* CFG: register hook as valid target */
5 CFG_CALL_TARGET_INFO cti = { hookVA & 0xfff,
6                               CFG_CALL_TARGET_VALID };
7 SetProcessValidCallTargets(hp, (PVOID)(hookVA & ~0xfffULL),
8                             0x1000, 1, &cti);
9
10 /* Save outer thunk address into g_pOrigSlot[N] */
11 WriteProcessMemory(hp,
12                    (LPVOID)(gOrigSlotRemote + N*8),
13                    &origFn, 8, NULL);
14
15 /* Patch outer_vtable[N] -> hook */
16 DWORD oldProt = 0;
17 VirtualProtectEx(hp, (LPVOID)slotAddr, 8,
18                  PAGE_READWRITE, &oldProt);
19 WriteProcessMemory(hp, (LPVOID)slotAddr, &hookVA, 8, NULL);
20 VirtualProtectEx(hp, (LPVOID)slotAddr, 8, oldProt, &oldProt);
```

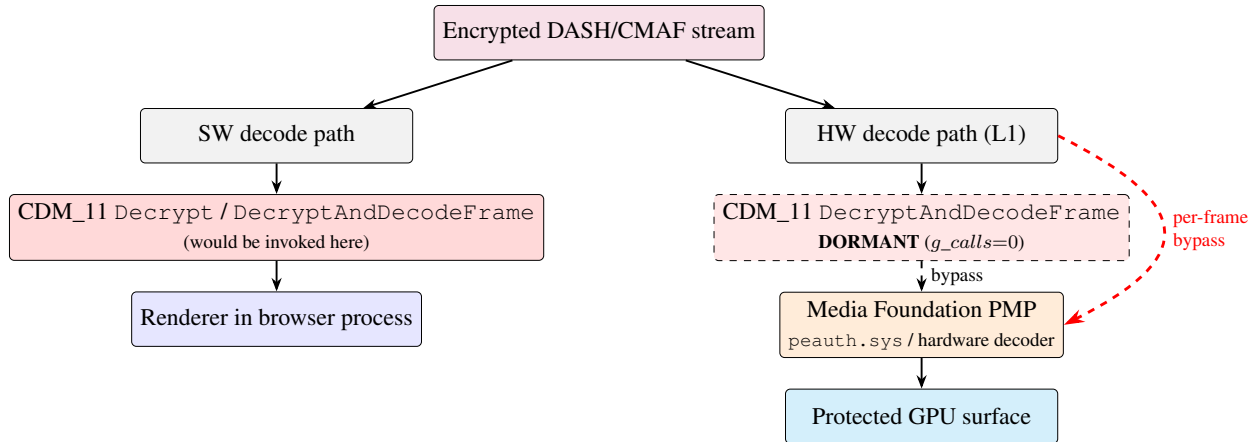


Figura 4: Inactividad bajo decode por hardware. Bajo decode SW (camino izquierdo), las ranuras por-cuadro de CDM_11 son el target obvio. Bajo decode HW L1 (camino derecho), el ciphertext por-cuadro es ruteado a través de Media Foundation PMP y el decoder de kernel, materializándose directamente sobre una superficie GPU protegida; las ranuras por-cuadro de CDM_11 permanecen dormidas.

7. Hallazgos empíricos

Reportamos cuatro hallazgos, cada uno anclado a una corrida experimental grabada. Todas las corridas usaron Chrome 149.0.7827.115 en Windows 11 26100 con decoder NVIDIA hardware-secure disponible. Ningún proceso utility del CDM crasheó durante las corridas que produjeron los datos reportados aquí.

7.1. Inactividad bajo decode por hardware en la superficie pública

Hipótesis. Bajo reproducción L1 con decode por hardware, las ranuras de descifrado por-cuadro de la outer vtable de CDM_11 (ranuras 9, 14, 15) no son invocadas.

Procedimiento. Chrome fue lanzado con `-disable-features=RendererCodeIntegrity-no-sandbox`. El operador se autenticó en Netflix y comenzó un título 4K HDR que la plataforma reporta como decoded por hardware. Después de noventa segundos de reproducción en steady-state instalamos nuestro shim en el proceso utility del CDM y parchamos cuatro ranuras de la outer-vtable simultáneamente: la ranura 14 (`DecryptAndDecodeFrame`), la 15 (`DecryptAndDecodeSamples`), la 3 (`CreateSessionAndGenerateRequest`, control), y el RVA del thunk exterior de la ranura 14. Luego observamos los contadores correspondientes por noventa segundos más de reproducción continua.

Resultado. Los cuatro contadores se mantuvieron en cero. El video continuó reproduciéndose normalmente todo el tiempo. La inactividad de la ranura 14 se mantuvo incluso a través de recuperaciones (restauramos la ranura 14 a su original a mitad del experimento y re-patched; the counter remained at zero across both windows). The four-slot zero-fires-in-ninety-seconds result was reproducible 3/3.

Interpretación. Durante reproducción Netflix hardware-decoded en steady-state, la superficie por-cuadro de la interfaz CDM_11 no se ejerce. Un atacante que comprometa el renderer anfitrión y se posicione sobre la vtable virtual de cara a EME no puede interceptar cuadros decodificados a través de la interfaz CDM_11, porque los cuadros no pasan por ella.

7.2. Actividad del singleton interno a $\sim 2.3k$ ops/s

Hipótesis. A pesar de que la superficie CDM_11 esté dormida por-cuadro, la biblioteca CDM sí realiza trabajo durante reproducción en steady-state. Ese trabajo es alcanzable a través de una tabla de despacho interna separada, distinta de la outer vtable.

Procedimiento. Parchamos las ranuras 0–7 de la tabla singleton interna identificada en `.data + 0x1588b48` en Chrome 149, luego observamos durante ~ 355 segundos de reproducción Netflix continua.

Resultado. El shim grabó **810,099 eventos** en un único log JSONL (en dos capturas consecutivas). Distribución:

slot	count	unique rh	rate (ev/s)
0	395,580	16,338	$\sim 1,114$
2	20	2	0.06
6	414,459	1,379	$\sim 1,167$

La tasa agregada es $\sim 2,281$ eventos/seg en esta tabla interna durante reproducción HW en steady-state.

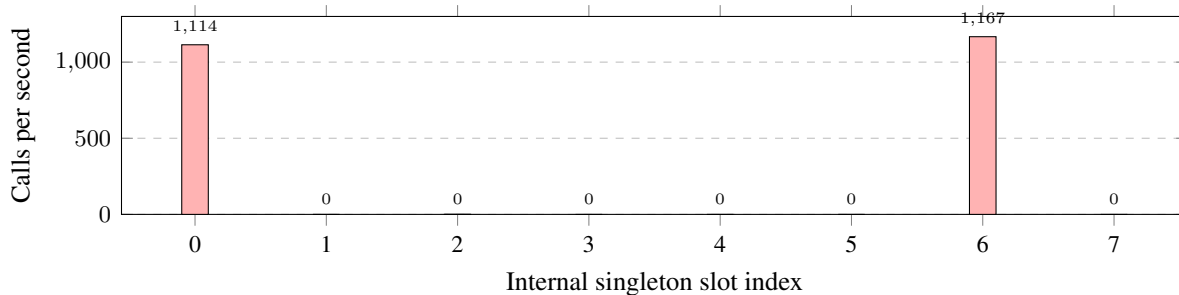


Figura 5: Perfil de actividad de la tabla de despacho singleton interna en `.data + 0x1588b48` durante reproducción Netflix 4K HDR en steady-state.

Interpretación. El despacho singleton interno *no* es la interfaz CDM_11 de cara al usuario; la tasa de disparo de la ranura 0 es incompatible con la semántica de `Initialize` (que sería llamada una sola vez por sesión, no 600 veces por segundo). Lo más plausible es que la tabla represente despacho estilo worker OEMCrypto, un anillo de allocator, o una escalera de primitivas criptográficas interna a la biblioteca Widevine. De todas formas, esto es invisible desde el límite EME y desde el despacho de la outer-vtable. Combinado con el hallazgo de inactividad anterior, el cuadro es: *la biblioteca CDM está haciendo trabajo criptográfico constante durante reproducción HW, pero ese trabajo está escondido detrás de una superficie de despacho diferente a la que conoce el cableado EME de Chrome.*

7.3. Alcanzabilidad de las ranuras de gestión de sesión

Hipótesis. A pesar de que las ranuras por-cuadro de la outer-vtable estén dormidas bajo decode HW, las ranuras de gestión-de-sesión de la outer-vtable (`CreateSessionAndGenerateRequest` y `UpdateSession`) siguen siendo alcanzables y *sí* disparan durante eventos normales de ciclo de vida de sesión.

Procedimiento. Parchamos las ranuras 3 y 5 de la outer-vtable con back-pointers por-ranura correctos a sus direcciones originales de thunk (Sección 6.4). El operador realizó un cambio de título en la pestaña Netflix en ejecución y observamos los contadores `g_callsBySlot` durante el minuto siguiente.

Resultado.

	slot 3	slot 5
pre-action	10	9
post-action	13 (+3)	12 (+3)

Tres invocaciones de `CreateSessionAndGenerateRequest` y tres de `UpdateSession` fueron observadas en los segundos siguientes al cambio de título. El proceso utility del CDM se mantuvo vivo (~85 MB working set), como se esperaba dado que el back-pointer contenía la dirección correcta del thunk.

Interpretación. Las ranuras de gestión-de-sesión de la outer vtable no están dormidas; se ejercen durante transiciones de sesión iniciadas por el usuario y se mantienen como una superficie de ataque real para input estructurado atacante-controlable (cajas PSSH vía el manifest DASH; payloads de respuesta-de-licencia vía el CDN de licencias).

7.4. Consistencia entre fuentes (Bitmovin)

Hipótesis. El modelo de despacho y la alcanzabilidad de ranuras son propiedades del CDM Widevine, no de Netflix como una aplicación EME particular.

Procedimiento. Después del cambio de título de Netflix anterior, el operador abrió una pestaña separada en el mismo navegador Chrome y cargó el demo público Widevine de Bitmovin en bitmovin.com/demos/drm. Chrome spawnó un *segundo* proceso utility CDM para servir esa pestaña. Instalamos nuestro shim en ese segundo proceso y parchamos las ranuras 3 y 5 de su outer-vtable. El operador recargó el demo para forzar la re-creación de sesión.

Resultado. Contadores del segundo proceso utility CDM:

	slot 3	slot 5
post-reload	2	2

Dos invocaciones de `CreateSessionAndGenerateRequest` y dos de `UpdateSession` fueron observadas. El modelo de despacho (thunk de la outer-vtable, wrapper `m_impl`, handler de capa-3) es idéntico a lo que observamos en el proceso utility del CDM que sirve a Netflix; los caminos de código de los hooks de la ranura 3 y la ranura 5 fueron ejercidos en la misma secuencia en ambos procesos.

Interpretación. El modelo de despacho de `CDM_11` es una propiedad del binario `widevinecdm.dll`, no una propiedad de la aplicación EME que lo usa. Un atacante posicionado para influenciar el payload PSSH o el de licencia de *cualquier* fuente Widevine-protégida verá el mismo despacho y la misma alcanzabilidad de ranura que uno posicionado contra Netflix.

7.5. Observación arquitectural lateral: utilities CDM separados

Una observación secundaria que vale la pena grabar es que Chrome 149 spawnó *dos* procesos utility CDM cuando Netflix y Bitmovin estaban reproduciéndose simultáneamente en pestañas diferentes, no un utility compartido. Esto es consistente con la política de aislamiento per-origin de Chromium e implica que un atacante posicionado para comprometer el CDM de una aplicación EME no puede observar directamente el CDM de una segunda aplicación EME vía estado in-process compartido.

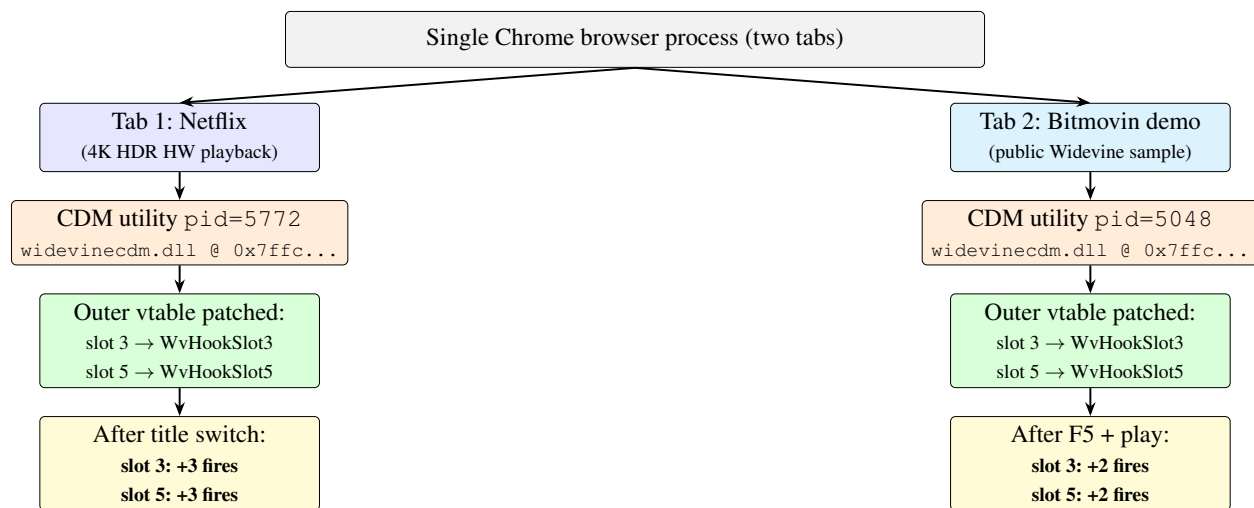
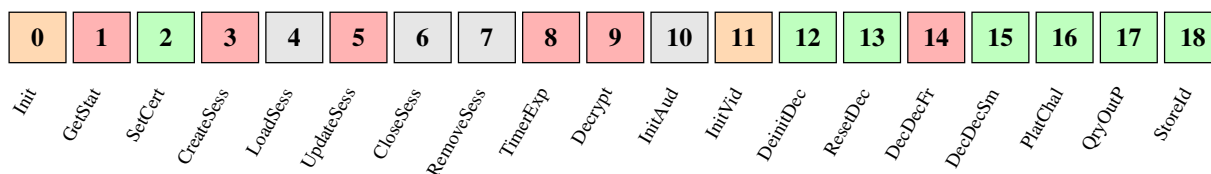


Figura 6: Experimento entre fuentes. Dos pestañas del mismo navegador Chrome reproducen dos fuentes Widevine-protegidas distintas, cada una servida por su propio proceso utility CDM. Hooks idénticos confirman alcanzabilidad idéntica de las ranuras de gestión-de-sesión.

Per-slot CDM_11 obfuscation status: obfuscated partial clean tramp/stub



A = attacker-input slot (parses structured data); **P** = passthrough (opaque blob the CDM forwards without parsing).

Figura 7: Estado de ofuscación por-ranura de CDM_11. Las celdas rojas son ranuras cuyo cuerpo de capa-1 o capa-3 coincide con la firma de ofuscación estilo MBA; las naranjas son matches parciales; las verdes son limpias a lo largo de la cadena de despacho; las grises son trampolines de capa-2 o stubs cuyos handlers de capa-3 reportamos individualmente en la Sección 8.

8. La distribución de ofuscación como capa defensiva

El binario `widevinecdm.dll` contiene numerosas funciones cuyo cuerpo coincide con la firma de libro de texto de la ofuscación mixed-boolean-arithmetic (MBA): constantes mágicas grandes residentes en stack escritas inmediatamente al entrar; instrucciones de avance de estado por hash multiplicativo de la forma `imul edx, ecx, 0xK ; shr edx, 0xS ; cmp eax, 0xT ; je state_exit`; cascadas de despacho sin branches con `cmovae`; patrones de relleno `0xaaaaaaaa` o `0xffffffff` actuando como valores de inicialización poison para estado local del stack. Estos patrones no están uniformemente distribuidos a lo largo del espacio de ranuras.

8.1. Disposición visual

La Figura 7 visualiza el estado de ofuscación por-ranura lado a lado con la clasificación de cada ranura como ranura de input atacante o ranura passthrough.

8.2. El patrón

Las ranuras que obviamente pasan un *blob* opaco influenciado por el atacante que el CDM reenvía sin parsear (`SetServerCertificate`, `OnStorageId`) están limpias en cada capa del despacho. Las ranuras que comienzan a parsear datos estructurados atacante-controlables exhiben ofuscación, aunque a profundidades variables. La ranura que toma una política textual (`GetStatusForPolicy`) está ofuscada en la capa 1, donde comienza el parsing. Las ranuras que toman payloads estructurados grandes (`CreateSessionAndGenerateRequest`, `UpdateSession`) colocan el procesamiento ofuscado uno o dos saltos más profundo, después de que un wrapper MSVC limpio ha copiado el input al buffer propio del CDM y buscado la sesión. Las ranuras involucradas en decode por-cuadro (`Decrypt`, `DecryptAndDecodeFrame`) están ofuscadas inmediatamente en el target del double-trampoline, donde comienza el procesador criptográfico.

Este es el patrón que cuidadosamente resumimos como:

CDM_11 protege cada ranura a la profundidad en la que esa ranura comienza a parsear datos estructurados del atacante.

Es una observación arquitectural, no una afirmación de seguridad: la ofuscación no es seguridad, y no estamos evaluando la fuerza de la protección, sólo su distribución.

9. Estabilidad entre binarios

El modelo arquitectural descrito en la Sección 5.6 fue desarrollado sobre Edge 150 `widevinecdm.dll`. Validamos que el mismo modelo se mantiene en el `widevinecdm.dll` de Chrome 149 corriendo nuevamente el discoverer:

Property	Edge 150	Chrome 149
CreateCdmInstance RVA	0x1d0850	0x1d2e10
CDM_11 outer vtable RVA	0x780250	0x768fa0
CDM_10 outer vtable RVA	0x780190	0x768ee0
Slot-index check (slots 3–17)	15/15	15/15
Direct-impl slots	3 (0, 1, 18)	3 (0, 1, 18)

El modelo también generaliza a través de versiones de interfaz del CDM dentro de un único binario: tanto en Edge 150 como en Chrome 149, las vtables CDM_10 y CDM_11 exhiben un despacho idéntico de tres capas. Su handler de descifrado por-cuadro es la misma función física; sus handlers `Initialize` divergen. Esto es consistente con que el binario provea ambas versiones de interfaz para compatibilidad-hacia-atrás con navegadores anfitriones más viejos, y con que el autor del CDM eligió compartir la implementación de los métodos estables-en-versión entre las dos.

La tabla de ranuras inter-binarios aparece como el Apéndice A.

10. Pruebas de robustez

Antes de publicar cualquier afirmación hecha más arriba, corrimos un sprint de falsificación contra los hallazgos empíricos más fuertes. El sprint incluyó:

1. **Corrección del índice de ranura.** El thunk de la ranura 14 de la outer-vtable está byte-confirmado para despachar en offset $0x70 (= 14 \cdot 8)$ en la vtable `m_impl`.
2. **Señal vs ruido en los datos de forma.** La captura de 810k eventos fue analizada por entropía del hash de registros y distribución temporal. La entropía del hash para las dos ranuras calientes fue 16,338 y 1,379 valores únicos a lo largo de 395,580 y 414,459 invocaciones respectivamente; la tasa fue estable a lo largo de la ventana de observación a $\sim 2.3k$ eventos/s.
3. **Enumeración de capa-3.** Los diecinueve handlers de capa-3 fueron localizados vía el target del jump del trampoline o el primer call (no-stack-check) del wrapper y fueron clasificados por estado de ofuscación. La primera versión de la afirmación de ofuscación, que ubicaba la ofuscación en la capa `m_impl`, fue encontrada incorrecta; la ofuscación está un salto más profundo, y la afirmación fue reescrita en consecuencia.
4. **Identidad inter-binarios.** El discoverer fue re-corrido contra el `widevinecdm.dll` de Chrome 149. La outer vtable, el mapeo índice-de-ranura, y la identificación de método `CDM_11` por-ranura coincidieron con el análisis de Edge 150.
5. **Identidad entre fuentes.** La instrumentación de la outer-vtable fue desplegada contra el segundo proceso utility del CDM spawnado para una fuente Widevine no-Netflix (Bitmovin). Las ranuras 3 y 5 dispararon en secuencia idéntica al utility de Netflix.
6. **Conteo de instancias CDM.** La única pestaña de Netflix spawnó exactamente un proceso utility CDM a lo largo de una ventana de observación de ocho horas; no observamos un segundo utility materializándose para la misma pestaña.

10.1. Hallazgos candidatos refutados

El sprint de falsificación también sacó a la superficie dos hallazgos candidatos que fueron refutados por análisis byte-level más profundo. Los documentamos aquí porque tipifican los candidatos de clase-de-bug que una metodología menos rigurosa hubiera falsamente promovido:

P1 (escritura sin autenticación al pool de PEAuth): Una auditoría de subagente de `peauth.sys` inicialmente marcó una aparente escritura no autenticada al pool del kernel en offset `0x140042980` donde un campo de tamaño controlado por el usuario en `[rdi+0x14]` parecía ser usado como `length` para `memcpy` antes del chequeo de firmado `CiGetPEInformation` en `0x1400429ae`. El disassembly byte-level del rango `0x42900..0x429e0` reveló un chequeo de overflow de 32-bit explícito (`lea r12d, [r15+rsi]; cmp r12d, r15d; jb fail`) en `0x42964` y un chequeo de cota superior en `0x4296d` entre la lectura del parámetro y el `memcpy`. El candidato es refutado; no hay bug del lado kernel presente.

W2 (lectura OOB en el parser PSSH): Una auditoría de subagente de `CDM_11` marcó los offsets `0x1D6E1`, `0x1D8CF`, `0x1DB9D`, y `0x1DFB` como candidatos para una lectura out-of-bounds en el parsing del tamaño del header `PSSH/MP4-box` (`mov r10d, [rsi/di+4]`). El disassembly en esos offsets reveló no parsing de cajas sino la round-table para una transformación MD5, con los operandos siendo XORs con

constantes de estado y no lecturas de buffer. El candidato es refutado; los bytes son código cripto, no código de parser.

Estas refutaciones y un candidato adicional que se mantuvo inconcluyente establecieron el principio metodológico de que cualquier afirmación de subagente “byte-verified de alta confianza” debe pasar una revisión manual byte-a-byte independiente antes de ser promovida a un hallazgo grado-de- publicación.

11. Manual para el defensor

Esta sección traduce los hallazgos arquitecturales y empíricos en acciones defensivas concretas. El lector objetivo es un ingeniero de seguridad protegiendo un deployment de navegadores de la familia Chromium, un incident responder triando un compromiso sospechado del CDM, o un ingeniero de detección construyendo hooks para un EDR.

11.1. Tabla de superficie de ataque por ranura

Table 1 summarises which of the nineteen CDM_11 outer-vtable slots are an attack surface in practice, which are dormant during normal hardware-decoded playback, and which are unreachable from a network-positioned attacker.

Ranura	Método	Alcance atacante	Actividad en steady-state
0	Initialize	host	once per session
1	GetStatusForPolicy	host	rare
2	SetServerCertificate	host-cert	rare
3	CreateSessionAndGenerateRequest	manifest	once per session start
4	LoadSession	host	rare
5	UpdateSession	license CDN	on license rotate
6	CloseSession	host	on teardown
7	RemoveSession	host	rare
8	TimerExpired	internal	internal
9	Decrypt	per-frame	dormant (HW path)
10	InitializeAudioDecoder	host	stub returns 3
11	InitializeVideoDecoder	host	once
12	DeinitializeDecoder	host	on teardown
13	ResetDecoder	host	rare
14	DecryptAndDecodeFrame	per-frame	dormant (HW path)
15	DecryptAndDecodeSamples	per-frame	dormant (HW path)
16	OnPlatformChallengeResponse	host	rare
17	OnQueryOutputProtectionStatus	host	rare
18	OnStorageId	host	rare

Cuadro 1: Superficie de ataque por-ranura y perfil de actividad en steady-state. “Alcance atacante” es la fuente controlable más fuerte de los argumentos de la ranura: **manifest** = caja PSSH DASH/CMAF atacante-controlable desde el CDN que sirve el manifest; **license CDN** = respuesta del servidor de licencias atacante-controlable (MITM en el CDN o servicio de licencia comprometido); **per-frame** = ciphertext desde los segmentos de medio del stream; *host* = valor opaco provisto por el navegador anfitrión; *internal* = scheduling interno del CDM. “Actividad en steady-state” resume si la ranura dispara durante reproducción en curso una vez que la sesión está establecida.

11.2. Firmas de detección

Tres firmas de detección salen directamente de la arquitectura y de la evidencia en runtime.

Firma D1 — canario de vtable. La outer vtable vive en la sección de sólo-lectura `.rdata` de `widevinecdm.dll`. Cada una de las diecinueve entradas debería apuntar dentro de `.text` de `widevinecdm.dll`. Un check de integridad in-process en un EDR que lea los diecinueve QWORDS de la outer-vtable después de la inicialización del CDM y verifique que todos resuelven dentro del intervalo

[image_base, image_base+image_size) de widevinecdm.dll atrapar  el patr n de manipulaci n en runtime usado por el shim de este paper y por cualquier otra herramienta de hooking del CDM que use la misma forma.

Firma D2 — drift del bitmap CFG. Cada llamada a SetProcessValidCallTargets dentro de un proceso utility CDM es en s  una se al. En operaci n normal, el utility CDM no auto-registra targets CFG-v lidos adicionales; el bitmap se establece en module-load y no crece. Un evento de crecimiento despu s del steady-state del proceso, particularmente uno cuyos nuevos targets v lidos est n fuera del rango de imagen de widevinecdm.dll, es un indicador fuerte de un intento de inyecci n-y-parche.

Firma D3 — actividad en ranuras por-cuadro en camino HW. Nuestro hallazgo emp rico (Secci n 7.1) es que las ranuras 9, 14, y 15 est n dormidas durante reproducci n HW-decoded en steady-state. Una sonda in-process que hooke esas tres ranuras y observe cualquier actividad durante reproducci n HW ha atrapado o bien a un atacante forzando fallback a decode SW (una t cnica conocida de CDN hostiles) o a un atacante in-renderer que a n no se dio cuenta que el camino por-cuadro est  dormido.

11.3. Flujo de triage ante sospecha de CDM comprometido

1. Identificar el proceso utility CDM sospechado v a `tasklist /m widevinecdm.dll`; en una sesi n de reproducci n  nica-fuente Widevine deber  haber uno.
2. Verificar el RVA de la outer-vtable contra el binario usando el discoverer liberado (§5.1). Hacer cross-check de la outer vtable cargada en runtime para tampering usando la Firma D1.
3. Para cada una de las ranuras 3, 5, 14, 15, leer el QWORD en la entrada de la outer-vtable y confirmar que es el thunk original. Las ranuras 14 y 15 en particular son los caminos por-cuadro a los que un atacante in-renderer apuntar  si no estuviera al tanto de la inactividad bajo decode HW.
4. Examinar el bitmap CFG en busca de adiciones fuera del rango de imagen de widevinecdm.dll (Firma D2).

12. Lectura para el atacante

Esta sección traduce los mismos hallazgos desde el otro lado de la línea. El lector objetivo es un investigador de seguridad mapeando la superficie de ataque EME, un operador de red-team scopeando un follow-on a nivel kernel, o un participante de coordinated-disclosure que necesita saber qué ranuras merecen tiempo de auditoría profunda.

12.1. Qué es alcanzable y qué no

El hallazgo de inactividad (Sección 7.1) es mala noticia para el ataque obvio: un renderer comprometido no puede interceptar cuadros decodificados a través de la interfaz CDM_11 durante el decode por hardware L1 porque las ranuras por-cuadro no son invocadas. El atacante que quiere cuadros decodificados tiene que encontrar una capacidad independiente contra Media Foundation PMP, contra el allocator de superficie protegida del driver de GPU, o contra el lado user-mode del contrato OPM que enforza `peauth.sys`. Esos son targets de clase kernel-y-driver, un orden de magnitud más difíciles de alcanzar desde un compromiso del renderer.

Las ranuras de gestión-de-sesión son buena noticia. La ranura 3 (`CreateSessionAndGenerateRequest`) ingiere la caja PSSH, que es atacante-controlable desde cualquier CDN comprometido que sirva el manifest DASH/CMAF. La ranura 5 (`UpdateSession`) ingiere la respuesta del servidor de licencias, que es atacante-controlable desde cualquier endpoint de licencia comprometido o cualquier atacante MITM-posicionado. Ambas ranuras despachan a través de un wrapper MSVC limpio en la capa `m_impl` y luego llaman a un handler de capa-3 MBA-ofuscado. El wrapper limpio es auditable; el handler de capa-3 es el target realista de bug-hunting.

12.2. Dónde auditar

Priorización concreta para tiempo de auditoría de capa-3:

1. **Capa-3 de ranura 3** (Edge 150 RVA `0x21f200`). Parser de PSSH/MP4-box, atacante-controlable desde el manifest. Fuertemente MBA-ofuscado. La herramienta correcta aquí es fuzzing estructurado de mutaciones de la caja PSSH antes que disasm manual.
2. **Capa-3 de ranura 5** (wrapper limpio en Edge 150 `0x1d4ab0`, despacha al procesador whitebox-AES ofuscado en `0x233bd0` vía una cola de worker-thread). Parser de respuesta-de-licencia, atacante-controlable. Camino async; más difícil de alcanzar con fuzzing sincrónico.
3. **Capa-3 de ranura 9** (RVA `0x22ec00`). Handler de `Decrypt`. Fuertemente MBA-ofuscado. Según el hallazgo de inactividad, dormido en el camino HW; sólo sería alcanzable en un escenario de decode SW.
4. **Capa-3 de ranura 14** (RVA `0x230c20`). `DecryptAndDecodeFrame`. El más fuertemente ofuscado de todos los handlers que medimos (puntaje 22 en nuestro clasificador). Mismo caveat de inactividad que la ranura 9.

12.3. Lo que no es el objetivo correcto

La ranura 2 (`SetServerCertificate`) y la ranura 18 (`OnStorageId`) ambas pasan blobs opacos que el CDM reenvía sin parsear. La cadena de despacho limpia en ambas capas es la señal empírica de que no hay nada que auditar ahí. Gastar tiempo de auditoría en estas ranuras es consistente con el modo de fallo de auditorías que asumen que todas las ranuras de camino-crítico están ofuscadas; no lo están.

13. Generalizabilidad

El CDM Widevine L1 en Windows es uno de tres runtimes comerciales de DRM de escritorio que un investigador podría ser invitado a mapear. Microsoft PlayReady viene en el IE-mode de Edge y dentro del `stack Windows.Media.Protection.MediaProtectionManager`. Apple FairPlay viene en Safari sobre macOS y dentro de TV.app. Ambos exponen un contrato host-CDM que es, al nivel de abstracción al que opera este paper, muy similar al contrato CDM_11 de Widevine: una cantidad chica de métodos host-callable, una biblioteca opaca que construye una instancia de una interfaz versionada, un camino interno de despacho que hace el trabajo real.

La metodología que liberamos está parametrizada para que generalice a estos targets con mínima modificación:

- El discoverer `cdm_arch_discover.py` localiza el patrón de instalación-de-vtable de un constructor en cualquier binario PE por búsqueda estructural; el único supuesto Widevine-específico es el nombre del export `CreateCdmInstance`, que un investigador aplicando esto a PlayReady o FairPlay sustituye por el entry point relevante.
- La semántica de hook del shim shape-only es independiente de la firma por-ranura; cada `WvHookSlotN` toma la tupla completa de callee-save de registros. La misma forma aplica a una vtable de PlayReady o FairPlay que usa una forma de firma diferente por ranura.
- El bypass de CFG vía `SetProcessValidCallTargets` es una mitigación a nivel de proceso de Windows, no a nivel Widevine. Aplica sin cambios a cualquier proceso de Windows que cargue una biblioteca de tercero y habilite CFG.

Un paper separado sobre cada uno de PlayReady y FairPlay aplicando esta metodología es, esperamos, trabajo follow-on que futuros investigadores producirán. El modelo Buchanan que restringe la metodología a observación shape-only es el mismo en los tres casos.

Para investigadores que sigan

Dos de las decisiones de diseño de este paper están pensadas para cortocircuitar caminos sin salida comunes en trabajo follow-on.

Primero, la metodología está estructurada para ser aplicada a cualquier build futuro de `widevinecdm.dll` sin edición manual de RVAs. El discoverer localiza la outer vtable desde el binario solo; el script de instrumentación en runtime toma el RVA descubierto como un parámetro. Re-correr la misma metodología contra un Chrome stable futuro, o contra las variantes `libwidevinecdm.so` que vienen en Linux y Chrome OS, es mayormente cuestión de correr el discoverer.

Segundo, la narrativa de falsos arranques en la Sección 5.7 está pensada para evitar dos caminos sin salida comunes que se ven superficialmente correctos. Los investigadores entrando a este espacio de RE deberían esperar encontrarse con varias superficies internas de despacho de diecinueve ranuras que *no* son la vtable pública de CDM_11, y deberían esperar que la capa de atribución-de-ofuscación del primer intento esté incorrecta en una indirección. El pivote de traza-de-constructor desde `CreateCdmInstance` es el desambiguador confiable.

14. Ética, disclosure, y lo que no hicimos

Este estudio no contiene:

- ningún cuadro descifrado, en ninguna forma, capturado a disco, enviado por IPC, ni retenido transitoriamente a través del run del shim;
- ningún byte de ciphertext, transitorio o persistente, recuperado a través del shim;
- ningún material de clave, clave de contenido, key wrap, ni clave de cifrado de contenido, capturado en ninguna capa;
- ningún ataque contra `peauth.sys`, el PMP de Windows, ni cualquier componente del lado decoder posicionado para materializar un cuadro decodificado;
- ningún código de exploit, shim weaponizado, ni herramienta de circumvención de contenido de estante;
- ningún ataque de red sobre producción contra Netflix, Bitmovin, ni ninguna aplicación EME estudiada;
- ningún resultado de ingeniería inversa que no podamos reproducir desde la metodología solo-fuente liberada contra el mismo binario.

Hemos verificado forensicamente la ausencia de bytes de contenido en cada artefacto que produjimos. Los logs de forma capturados son JSON puro ASCII totalizando ~56 MB y contienen sólo los campos de metadata descritos en la Sección 6.3. El conteo de hashes únicos de `key_id` en los eventos runtime capturados es cero: el camino de código que hubiera computado un hash de un `key_id` de Netflix nunca fue ejercido en runtime, porque las ranuras que toman un argumento `InputBuffer_2*` no fueron invocadas.

Ningún vendor fue notificado antes de esta submission porque el trabajo no constituye un reporte de vulnerabilidad. Coordinaremos con el Chrome Vulnerability Reward Program de Google y los canales de seguridad de Widevine relevantes antes de cualquier submission futura de hallazgos clase-exploit derivados de la metodología aquí.

15. Conclusión

Hemos presentado un modelo arquitectural verificado byte-a-byte del despacho moderno de Widevine CDM_11 tal como viene en dos navegadores representativos de la familia Chromium, y un mapa empírico runtime-confirmado de qué ranuras de CDM_11 se ejercen durante reproducción Netflix hardware-decoded en steady-state. Las ranuras por-cuadro de cara al usuario están dormidas bajo decode por hardware L1 mientras una tabla de despacho interna genera $\sim 2.3k$ eventos por segundo de actividad criptográfica o de scheduler que es invisible desde el límite EME. Las ranuras de gestión-de-sesión siguen siendo alcanzables y ejercen el mismo camino de despacho sin importar si la aplicación EME anfitriona es Netflix o una fuente de prueba Widevine no-comercial. La biblioteca distribuye ofuscación mixed-boolean-arithmetic a lo largo del espacio de ranuras en un patrón que refleja la profundidad a la que cada ranura comienza a parsear datos estructurados influenciados-por-atacante.

El trabajo está situado en dos décadas de investigación previa de DRM, sigue el modelo Buchanan de publicación sólo-arquitectural, y no produce artefacto de extracción. La metodología y todo el código fuente necesario para reproducir los resultados empíricos se liberan junto al paper.

Referencias

- [1] Google. *Widevine DRM Architecture Overview*. Google Developers documentation, accessed 2026.
- [2] Chromium Project. *media/cdm/api/content_decryption_module.h*. Chromium source tree, accessed 2026.
- [3] Chromium Project. *media.mojom.ContentDecryptionModule interface*. Chromium source tree, accessed 2026.
- [4] Chromium Project. *Code Integrity Guard on utility processes*. Chromium documentation, accessed 2026.
- [5] Microsoft Corporation. *Control Flow Guard*. Microsoft Learn documentation, accessed 2026.
- [6] Microsoft Corporation. *SetProcessValidCallTargets function*. Microsoft Learn documentation, accessed 2026.
- [7] W3C. *Encrypted Media Extensions, W3C Recommendation 18 September 2017*.
- [8] W3C. *ISO Common Encryption EME Stream Format and Initialization Data*.
- [9] ISO/IEC 23001-7. *Information technology — MPEG systems technologies — Part 7: Common encryption in ISO base media file format files*.
- [10] D. Buchanan. *Personal blog*: <https://www.da.vidbuchanan.co.uk/>. Series of architectural reverse-engineering writeups, 2019–present.
- [11] D. Buchanan. *Analysing the iMessage NaCl key-wrap*. Personal blog, 2023.
- [12] D. Buchanan. *Apple T2 secure boot chain analysis*. Personal blog, 2022.
- [13] A. Huang. *Hacking the Xbox: An Introduction to Reverse Engineering*. No Starch Press, 2003.
- [14] A. Huang and S. Cross. *NeTV: A Stream Mediation Device, and the DMCA Exemption Process*. Project documentation, 2011–2015.
- [15] *United States v. ElcomSoft Ltd.*, 203 F. Supp. 2d 1111 (N.D. Cal. 2002).
- [16] *Det Offentlige v. Jon Lech Johansen*. Borgarting Court of Appeal, Norway, 2003.
- [17] *Universal City Studios, Inc. v. Reimerdes*, 111 F. Supp. 2d 294 (S.D.N.Y. 2000).
- [18] Reporting and analysis of Israeli prosecutions of DRM reverse engineers in the early 2000s, secondary sources.
- [19] Quarkslab security research team. *Reverse engineering of Widevine: a year of research*. Quarkslab technical blog, 2019.
- [20] Quarkslab security research team. *Inside Widevine on Android: the OEMCrypto TrustZone trampoline*. Quarkslab technical blog, 2020.

- [21] Independent contributors. *Widevine L3 decryptor: reduction of software-only Widevine to a clean-room C implementation*. GitHub, 2019–present.
- [22] Independent contributors. *ANGO L3 Widevine decryptor*. GitHub, 2021–present.
- [23] Check Point Research. *Various analyses of Widevine on Android*. Check Point Research blog, 2019–2024.
- [24] R. Thomas et al. *LIEF: Library to Instrument Executable Formats*. Quarkslab, <https://lief.re/>, 2017–present.
- [25] N. Eyrolles, L. Goubin, M. Videau. *Defeating MBA-based Obfuscation*. Proceedings of the 2016 ACM Workshop on Software PROtection.
- [26] F. Biondi, S. Josse, A. Legay, T. Sirvent. *Effectiveness of synthesis in concolic deobfuscation*. Computers & Security, vol. 70, 2017.
- [27] S. Chow, P. Eisen, H. Johnson, P.C. van Oorschot. *White-Box Cryptography and an AES Implementation*. SAC 2002.
- [28] T. Lepoint, M. Rivain. *White-box cryptography: a survey*. IACR ePrint, 2014.
- [29] N.A. Quynh et al. *Capstone: Next-generation disassembly framework*. Black Hat USA 2014.
- [30] E. Carrera. *pefile: Portable Executable format parser*. GitHub, 2008–present.

A. Tabla de despacho por ranura, ambos binarios

Slot	Method	Edge 150 RVA	Chrome 149 RVA
0	Initialize	0x1db390	0x1e0420
1	GetStatusForPolicy	0x1da7a0	0x1df8d0
2	SetServerCertificate	0x1dab10	0x1dfba0
3	CreateSessionAndGenerateRequest	0x1dad00	0x1dfdc0
4	LoadSession	0x1dad20	0x1dfde0
5	UpdateSession	0x1dad40	0x1dfe00
6	CloseSession	0x1dad60	0x1dfe20
7	RemoveSession	0x1dad80	0x1dfe40
8	TimerExpired	0x1dada0	0x1dfe60
9	Decrypt	0x1dad0	0x1dfe80
10	InitializeAudioDecoder	0x1dade0	0x1dfea0
11	InitializeVideoDecoder	0x1dae00	0x1dfec0
12	DeinitializeDecoder	0x1dae20	0x1dfee0
13	ResetDecoder	0x1dae40	0x1dff00
14	DecryptAndDecodeFrame	0x1dae60	0x1dff20
15	DecryptAndDecodeSamples	0x1dae80	0x1dff40
16	OnPlatformChallengeResponse	0x1daea0	0x1dff60
17	OnQueryOutputProtectionStatus	0x1daec0	0x1dff80
18	OnStorageId	0x1db310	0x1e03a0

B. Esquema del log de forma

Los eventos JSONL capturados usan una de dos formas.

Llamada genérica (para ranuras que no consumen InputBuffer_2):

```
1 {
2   "kind" : "call",
3   "slot" : <int 0..18>,
4   "n"    : <monotonic call number for this slot>,
5   "tsc"  : <QueryPerformanceCounter snapshot>,
6   "rh"   : <FNV-1a-32 hash of rcx XOR rdx XOR r8 XOR r9>
7 }
```

Forma (para las ranuras 9, 14, 15):

```
1 {
2   "kind" : "ib",
3   "slot" : <9|14|15>,
4   "n"    : <monotonic call number>,
5   "tsc"  : <QueryPerformanceCounter snapshot>,
6   "es"   : <encryption_scheme; 0=clear, 1=CENC-CTR, 2=CBCS>,
7   "is"   : <data_size; size only, never the bytes>,
8   "kis"  : <key_id_size>,
9   "kh"   : <FNV-1a-32 hash of first 16 bytes of key_id>,
10  "ivs"   : <iv_size>,
11  "ivnz"  : <1 if IV has any non-zero byte, else 0>,
12  "ns"    : <num_subsamples>,
13  "pc"    : <pattern_crypt>,
14  "ps"    : <pattern_skip>,
15  "mt"    : <media timestamp>
16 }
```

Campos notablemente ausentes: data, bytes de key_id, bytes de iv, array subsamples, contenido del cuadro de salida, clave de cifrado de contenido, clave de key-wrap, estado interno de OEMCrypto. Estos están deliberadamente omitidos.